Vue 3: Templates, Directivas y Renderización

En esta clase trabajaremos los conceptos con Vue por CDN. La sintaxis con <script setup> y archivos .vue se verá en la siguiente clase al crear un proyecto con npm/Vite.

NotebookLM

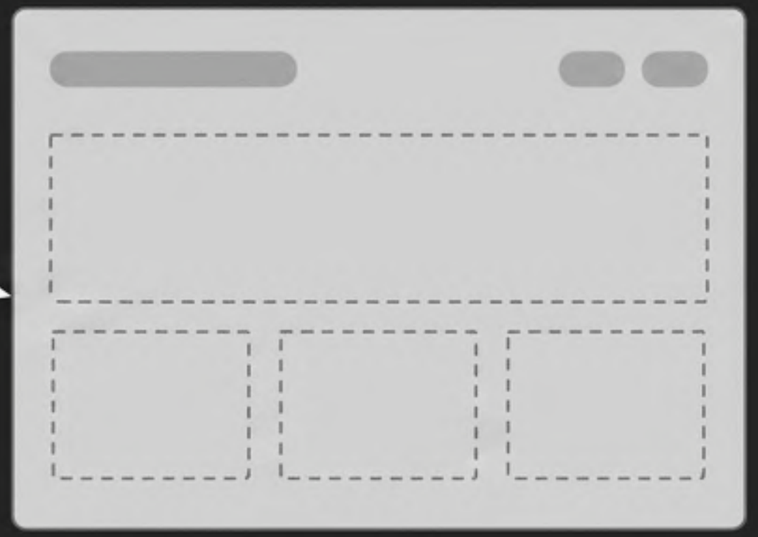
El Concepto: Templates y Reactividad

Un template en Vue representa la vista (HTML estándar mejorado). Declara cómo la interfaz de usuario (UI) debe verse basándose en el estado reactivo de la aplicación. Cuando este estado reactivo cambia, Vue se encarga de actualizar el DOM automáticamente.

Representa la Vista

```
<div class="clima-card">
  <!-- Los datos irán aquí -->
</div>
```

Basado en Estado Reactivo



NotebookLM

El Motor: Composition API y Refs

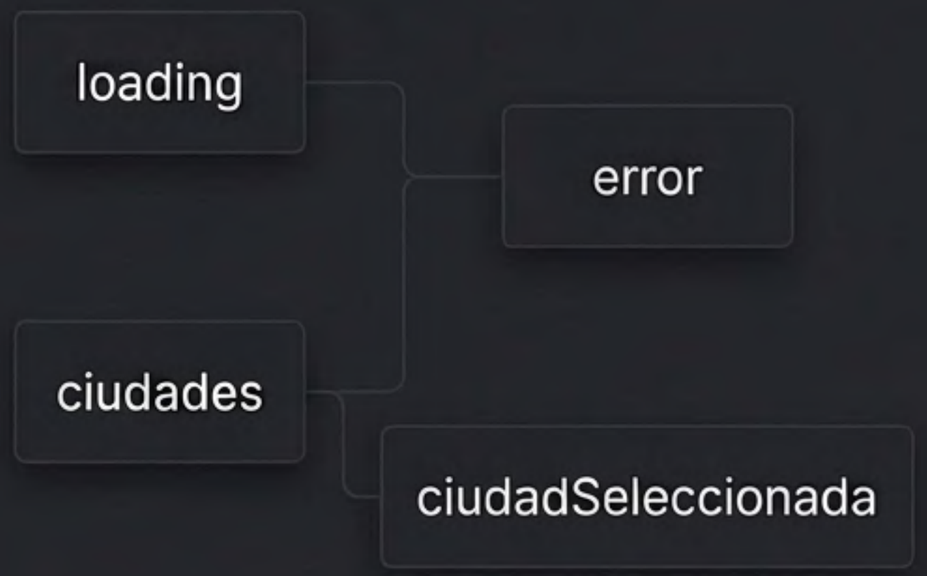
Usamos `<script setup>` y `ref()` para declarar variables reactivas. Si el valor de un `ref` cambia, la UI se vuelve a renderizar.

```
const { createApp, ref } = Vue

createApp({
  setup() {
    // Estado de nuestra app del clima
    const loading = ref(true)
    const error = ref(null)
    const ciudades = ref([])
    const ciudadSeleccionada = ref('')
    const climaHoy = ref(null)

    return {
      loading,
      error,
      ciudades,
      ciudadSeleccionada,
      climaHoy
    }
  }
})
```

Variables de Estado



En JavaScript accedemos/modificamos un ref con **.value**. En el template Vue lo desenvuelve automáticamente.

```
const ciudad = ref('Santiago')
ciudad.value = 'Valparaíso'
```

En template:

```
{{ ciudad }}
```

NotebookLM

Mostrando Datos: Interpolación

La sintaxis de bigotes {{ }} enlaza datos reactivos al texto del DOM. Permite mostrar datos y expresiones simples. La interpolación se usa para texto y expresiones simples, no para lógica compleja.

```
....
{{ ciudadSeleccionada }}
<div v-if="climaHoy">
  {{ climaHoy.temp }} °C
</div>
```

Bogotá

18°C

NotebookLM

Anatomía de una Directiva

Las directivas son atributos especiales de Vue (con prefijo `v-`) que modifican comportamiento, atributos, eventos o el renderizado del DOM.

Prefijo: v- Nombre: on Argumento: :mteven

```
<form v-on:submit.prevent="buscarClima">
```

Argumento: :submit Modificador: .prevent Valor: "buscarClima"

Equivalencia moderna (atajo):

```
<form @submit.prevent="buscarClima">
```

(Atajo para v-on:)

NotebookLM

Atributos Dinámicos: v-bind

v-bind (o su atajo `:`) enlaza atributos HTML a variables de JavaScript. Se usa cuando el valor del atributo viene desde JavaScript y no es texto fijo. Esencial para imágenes, enlaces y estados de botones.

```
<!-- Formato completo -->
<img v-bind:src="climaHoy.iconoUrl" v-bind:alt="climaHoy.descripcion">

<!-- Atajo (shorthand) -->
<img :src="climaHoy.iconoUrl" :alt="climaHoy.descripcion">

<button :disabled="loading">
  Buscar
</button>
```

NotebookLM

Clases y Estilos Reactivos

Vue mejora v-bind para class y style permitiendo usar objetos y arreglos.
Las clases se aplican si la condición es true.

```
1 <div
2   class="card"
3   :class="{
4     'bg-frio': climaHoy.temp < 10,
5     'bg-calor': climaHoy.temp > 25
6   }"
7   :style="{ fontSize: baseFontSize + 'px' }"
8 > {{ climaHoy.temp }}°C
```

class fija mantiene estilos base y :class agrega clases de forma condicional.

NotebookLM

Interacción del Usuario: v-model

v-model establece un enlace bidireccional (*two-way binding*) entre un elemento de formulario y una variable del estado. Sincroniza automáticamente los datos entre el input y la variable reactiva.

1. v-model en input

```
<input v-model="busqueda" placeholder="Buscar ciudad">
```

2. v-model con modificador .trim

```
<input v-model.trim="nombreUsuario" placeholder="Nombre">
```

3. v-model en select

```
<select v-model="ciudadSeleccionada">
  <option disabled value="">Selecciona una ciudad</option>
  <option>Bogotá</option>
  <option>Lima</option>
</select>
```

NotebookLM

C

Renderizando Listas: v-for y :key

En este curso usaremos siempre `:key` al renderi**zar listas** para que Vue identifique cada **elemento** de forma **estable**.

```
<select v-model="ciudadSeleccionada">
  <option
    v-for="ciudad in ciudades"
    :key="ciudad.id"
    :value="ciudad"
  >
    {{ ciudad.nombre }}, {{ ciudad.pais }}
  </option>
</select>
```

💡 Al guardar el objeto completo en el v-model, se puede acceder a propiedades adicionales como `ciudad.lat` y `ciudad.lon` para futuras consultas a la API.

NotebookLM

Manejo de Eventos: v-on

- **v-on** (o su atajo **@**) escucha eventos del DOM y ejecuta métodos de JavaScript.
- Los modificadores simplifican tareas comunes.
- Equivalencia moderna: **v-on:click**="forzarRecarga" **@click**="forzarRecarga"

Action-Trigger

Action-Trigger

```
<form @submit.prevent="buscarClima">
  <select v-model="ciudadSeleccionada" @change="cargarDatos">
    <option>...</option>
  </select>

  <button @click="forzarRecarga">
    Refrescar
  </button>
</form>
```

Función JS: forzarRecarga()

Evita que el formulario recargue la página.

NotebookLM

Renderizado Condicional: v-if, v-else-if, v-else

Estas directivas **crean o eliminan** elementos del DOM según una condición. Los bloques `v-else-if` y `v-else` deben ir **inmediatamente después** de un elemento con `v-if` o `v-else-if`.

```
<div v-if="loading" class="spinner">
  Cargando pronóstico...
</div>
```

Estado de Error

```
<div v-else-if="error" class="alerta">
  Error: {{ error.mensaje }}
</div>
```

Estado Correcto

```
<div v-else class="clima-dashboard">
  <!-- Contenido del Clima -->
</div>
```

NotebookLM

Alternando Visibilidad: v-show

v-show no destruye el elemento; solo cambia su display CSS.
Comparación breve:
v-if elimina/crea elementos.
v-show oculta/muestra elementos.

```
<button @click="mostrarDetalles = !mostrarDetalles">
  Ver más detalles
</button>

<div v-show="mostrarDetalles" class="panel-extra">
  <p>Humedad: {{ climaHoy.humedad }}%</p>
  <p>Viento: {{ climaHoy.viento }} km/h</p>
</div>
```

CSS Inyectado:
display: none

NotebookLM

Matriz de Decisión: v-if vs. v-show

Concepto Definitivo: Elegir la directiva correcta optimiza el rendimiento de la aplicación.

Característica	v-if	v-show
Mecánica	Crea/Destruye el nodo DOM	Alterna display: none CSS
Costo Inicial	Bajo (No renderiza hasta que la condición es verdadera)	Alto (Se renderiza desde el inicio)
Costo al Alternar	Alto (Crea/Destruye el elemento)	Bajo (Cambia display)
Uso Ideal	Estados raros (Login, Errores de API)	Alternancia frecuente (Menús, Acordeones)

Pro-Tip: Evita usar v-if y v-for en el mismo elemento. Filtra antes los datos o coloca el v-if en un contenedor padre.

NotebookLM

State Timeline

Setup (Loading State)	onMounted (Fetch API)	Template Update (Render Data)

Adelanto: onMounted

Este tema se verá con más detalle al trabajar ciclo de vida y consumo de APIs.

onMounted se ejecuta después de que el componente fue montado en el DOM.

PREVIEW

```
const { createApp, ref, onMounted } = Vue

createApp({
  setup() {
    // ... variables reactivas

    onMounted(async () => {
      try {
        // ... loading=true, fetch API, climaHoy=data
      } catch (err) {
        // ... error=err
      } finally {
        // ... loading=false
      }
    })
    // ... return
  }
}).mount('#app')
```

NotebookLM

Síntesis: La Arquitectura Vue 3

Clase actual: Vue por CDN + **createApp** + setup + template en HTML.

```
const { createApp, ref } = Vue

createApp({
  setup() {
    const mensaje = ref("Hello Vue")
    return { mensaje }
  }
}).mount("#app")
```

Siguiente clase: npm/Vite + archivos **.vue** + **<script setup>**.

Con npm/Vite separaremos esta lógica en main.js, App.vue y componentes .vue, utilizando una sintaxis más moderna y herramientas de compilación.

Takeaway

Vue conecta estado reactivo, template y eventos para construir interfaces dinámicas sin manipular el DOM manualmente.

NotebookLM